

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»



ПОЛИТЕХ

Санкт-Петербургский
политехнический университет
Петра Великого

Физико-механический институт
Высшая школа прикладной математики и вычислительной
физики

Отчёт по лабораторной работе №1

по теме:

Реализация протоколов Go-Back-N и Selective Repeat

по дисциплине:

Компьютерные сети

Выполнил:

Худина Анастасия Алексеевна

группа 5040102/50201

Принял:

к. ф.-м. н., доцент

Баженов Александр Николаевич

Санкт-Петербург
2026 г.

ОГЛАВЛЕНИЕ

Постановка задачи	3
Теория	4
1. ARQ и скользящее окно	4
2. Протокол <i>Go-Back-N</i>	6
3. Протокол <i>Selective Repeat</i>	8
4. Сравнение и метрики	10
Имитационная модель и реализация	13
Результаты эксперимента	17
1. Зависимость накладных расходов от вероятности ошибок	17
2. Зависимость коэффициента эффективности от вероятности ошибок	19
3. Зависимость времени передачи от вероятности ошибок	20
4. Сравнение с аналитическими оценками	22
Заключение	25
Список использованных источников	28

ПОСТАНОВКА ЗАДАЧИ

Необходимо построить имитационную модель системы передачи данных, состоящей из отправителя и получателя, которые обмениваются кадрами через канал с ошибками. Канал может терять кадры с заданной вероятностью p .

Передача должна быть реализована двумя способами: с помощью протокола Go-Back-N (GBN) и протокола Selective Repeat (SR). В модели можно менять два параметра: вероятность ошибки p и размер скользящего окна W .

Для каждой пары значений требуется передать фиксированное число кадров и посчитать, сколько всего передач потребовалось, что позволит измерить накладные расходы протокола.

Цель работы – исследовать, как накладные расходы зависят от вероятности ошибки и размера окна, и сравнить два протокола между собой: какой из них эффективнее расходует канал. Также, чтобы убедиться, что модель реализована правильно, результаты имитации необходимо сравнить с известными теоретическими формулами.

По итогам экспериментов необходимо построить графики зависимости накладных расходов от вероятности ошибки при разных размерах окна и на их основе сделать выводы о том, в каких условиях лучше применять каждый из протоколов.

ТЕОРИЯ

1. ARQ и скользящее окно

При передаче данных по реальному каналу связи невозможно гарантировать, что каждый отправленный кадр дойдёт до получателя. Кадры могут теряться из-за помех, затухания сигнала, перегрузки промежуточных узлов или других причин.

Для того чтобы обеспечить надёжную доставку данных в правильном порядке, поверх ненадёжного канала строится механизм подтверждений и повторных передач. Такой механизм получил название автоматического запроса повторной передачи (Automatic Repeat reQuest, ARQ). Идея ARQ состоит в том, что получатель после успешного приёма кадра отправляет отправителю подтверждение (ACK), а отправитель, не дождавшись подтверждения за отведённое время, повторяет передачу. Чтобы получатель мог отличить новый кадр от повторно переданного и восстановить правильный порядок, все кадры нумеруются. Таким образом, ARQ опирается на три базовых элемента: нумерацию кадров, подтверждения и повторные передачи по тайм-ауту.

Простейшая реализация ARQ – протокол с остановкой и ожиданием (Stop-and-Wait). Отправитель передаёт один кадр, запускает таймер и ждёт подтверждения. Только после получения ACK он отправляет следующий кадр. Если подтверждение не приходит за время тайм-аута, кадр передаётся заново. Такой протокол надёжен, но крайне неэффективен: канал простаивает всё то время, пока отправитель ожидает ACK, а это время складывается из задержки распространения сигнала туда и обратно (RTT) и времени обработки на приёмнике. Если пропускная способность канала велика, а RTT заметно, доля полезной загрузки канала оказывается ничтожной.

Чтобы повысить эффективность использования канала, применяется принцип конвейеризации: отправителю разрешается иметь несколько

неподтверждённых кадров «в полёте» одновременно. Это реализуется с помощью механизма скользящего окна (sliding window).

Окно – это диапазон порядковых номеров, которые отправитель имеет право передать, не дожидаясь подтверждения. Размер окна W задаёт максимальное число кадров, одновременно находящихся в пути. Все кадры в пространстве номеров делятся на четыре категории:

- 1) уже переданные и подтверждённые;
- 2) переданные, но ещё не подтверждённые (они находятся внутри окна);
- 3) те, которые разрешено передать, но они ещё не отправлены;
- 4) те, которые пока передавать нельзя.

Отправитель хранит две переменные: `send_base` – номер самого старого неподтверждённого кадра, и `next_seq_num` – номер следующего кадра, готового к отправке (см. рис. 1).

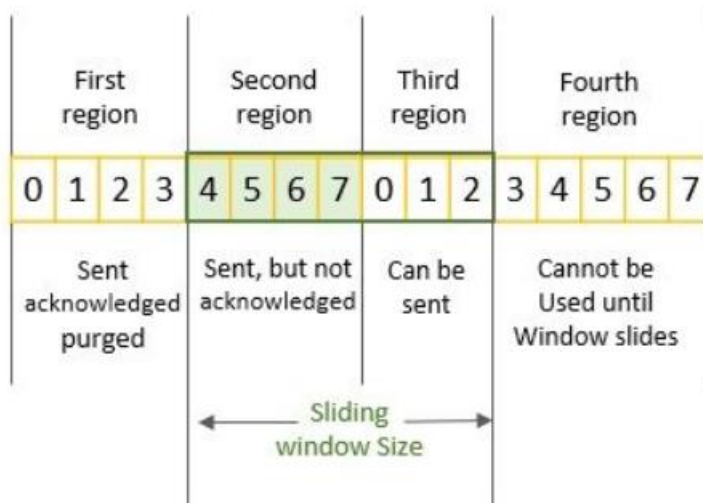


Рисунок 1. Схема состояний последовательности кадров при использовании скользящего окна [1].

Когда приходит подтверждение на кадр с номером `send_base`, левая граница окна сдвигается вправо, и отправитель получает право передать новые кадры. Если подтверждение приходит на более поздний кадр, поведение зависит от конкретной дисциплины ARQ: в одних протоколах такие подтверждения трактуются как накопленные, в других обрабатываются индивидуально. Именно в этом состоит главное различие

между протоколами Go-Back-N и Selective Repeat, которые рассматриваются ниже.

С размером окна связано важное ограничение. Если номер кадра занимает k бит, то пространство номеров содержит $M = 2^k$ различных значений, и после исчерпания номера нумерация идёт по кругу. Чтобы получатель не спутал новый кадр со старым, размер окна не должен превышать определённой величины. Для Go-Back-N допустимо $W \leq M - 1$, для Selective Repeat – $W \leq M/2$. Эти ограничения обеспечивают однозначность интерпретации номеров при любых перестановках и повторных передачах.

2. Протокол Go-Back-N

Протокол Go-Back-N (GBN), представленный на рис. 2, является одной из первых и наиболее простых конвейерных дисциплин ARQ. Его основная идея состоит в том, что получатель принимает кадры строго по порядку и использует накопленные подтверждения. Отправитель может отправить до W кадров, не дожидаясь АСК, но получатель не буферизует кадры, пришедшие вне очереди: если ожидается кадр с номером n , а приходит кадр с номером $n+1$, получатель его отбрасывает и повторно отправляет АСК для последнего корректно принятого кадра (в данной реализации – АСК с номером $n-1$, то есть подтверждение последнего кадра, дошедшего до приёмника по порядку). Такой АСК называется кумулятивным: он одновременно подтверждает все кадры с номерами до указанного включительно.

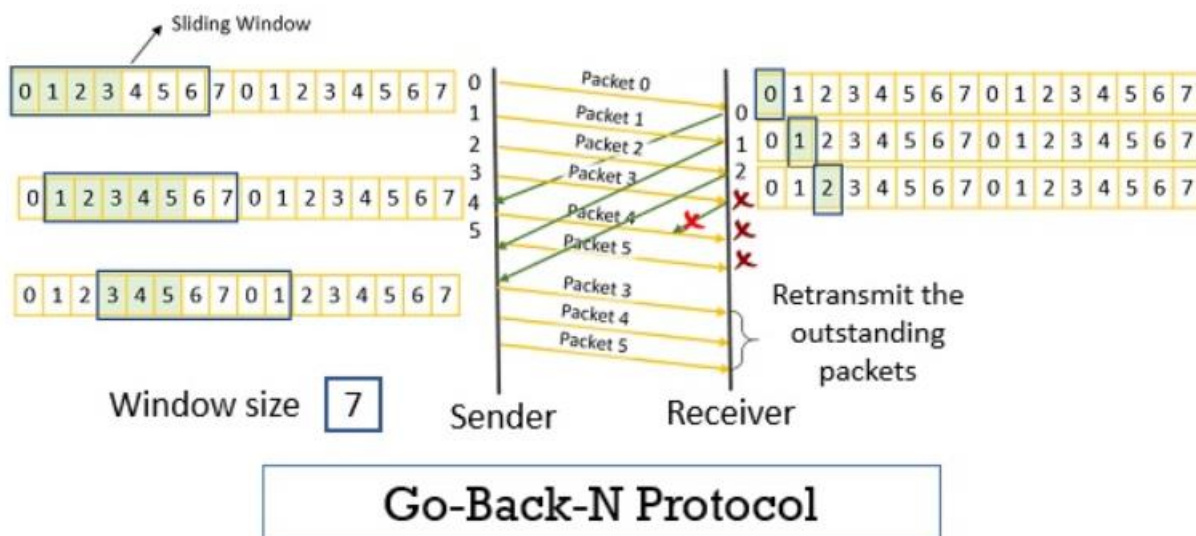


Рисунок 2. Диаграмма работы протокола Go-Back-N [1].

У отправителя GBN работает один таймер, привязанный к самому старому неподтверждённому кадру (`send_base`). Когда этот таймер истекает, отправитель повторно передаёт не только сам потерянный кадр, но и все кадры, начиная с него, вплоть до последнего отправленного (`next_seq_num-1`). В этом и состоит смысл названия протокола: отправитель «возвращается на N назад» и передаёт заново целую пачку кадров.

Именно поэтому GBN легко реализовать: получателю достаточно хранить один ожидаемый номер и не нужно держать буфер для кадров, пришедших вне очереди; отправителю нужен всего один таймер. Плата за простоту – избыточность передачи. Если потерялся один кадр, отправитель повторно передаёт все кадры, которые уже успели уйти в путь после него, даже если они были успешно приняты получателем. Чем больше размер окна и чем выше вероятность потери, тем больше объём повторно передаваемых данных.

Работа протокола GBN протекает следующим образом. Отправитель передаёт кадры с номерами 0, 1, ..., $W-1$, не дожидаясь подтверждений. Получатель принимает кадр с номером 0, отправляет АСК 0, ожидает кадр 1, и так далее, пока поток кадров идёт в правильном порядке. Если очередной кадр теряется, все последующие кадры, приходящие к

получателю, не соответствуют ожидаемому номеру и отбрасываются; получатель повторяет АСК последнего успешно принятого кадра. Отправитель, получив такой «устаревший» АСК, не сдвигает окно, а когда истекает таймер у `send_base`, повторяет всю пачку кадров, начиная с потерянного. Получатель, получив пропущенный кадр, принимает его и продолжает приём следующих кадров, которые теперь снова идут по порядку. Кумулятивные подтверждения при этом автоматически «сдвигают» окно отправителя через накопленный префикс.

Важно понимать, что GBN корректен только при выполнении условия $W \leq M - 1$. Если размер окна превысит это значение, при повторной передаче номера кадров могут совпасть с номерами новых кадров, и получатель не сможет отличить старый кадр от нового. Это ограничение напрямую связано с тем, что в GBN окно отправителя может охватывать почти всё пространство номеров без риска неоднозначности, но не может охватить его полностью.

Таким образом, Go-Back-N обеспечивает надёжную передачу при минимальных требованиях к состоянию получателя и отправителя, но расплачивается за это повышенным объёмом повторных передач. Именно поэтому этот протокол выгоден в каналах с малой вероятностью ошибок или при небольших размерах окна: тогда вероятность потерять кадр внутри окна невелика, и избыточность передачи остаётся приемлемой. При высоких p и больших W эффективность GBN резко падает – что и будет продемонстрировано в экспериментальной части работы.

3. Протокол Selective Repeat

Протокол Selective Repeat (SR), представленный на рис. 3, устраняет главный недостаток Go-Back-N – избыточную повторную передачу кадров, которые уже были успешно приняты получателем. Если в GBN потеря одного кадра приводит к повторной передаче всей последующей пачки, то в SR отправитель повторяет только тот кадр, на который не пришло

подтверждение. Такой подход существенно экономит пропускную способность канала, но требует более сложной логики как на стороне отправителя, так и на стороне получателя.

Ключевое отличие SR от GBN состоит в том, что получатель буферизует кадры, пришедшие вне очереди. Если ожидается кадр с номером n , а приходит кадр с номером $n+2$, получатель не отбрасывает его, а сохраняет в буфере и отправляет индивидуальное подтверждение ACK именно за номер $n+2$. Когда наконец приходит пропущенный кадр n , получатель доставляет все накопленные кадры в правильном порядке выше по стеку и сдвигает ожидаемый номер сразу за всю непрерывную последовательность. То есть вместо кумулятивного подтверждения, как в GBN, здесь используется подтверждение на каждый конкретный кадр – ACK(n) означает «кадр с номером n получен», а не «получены все кадры до n включительно».

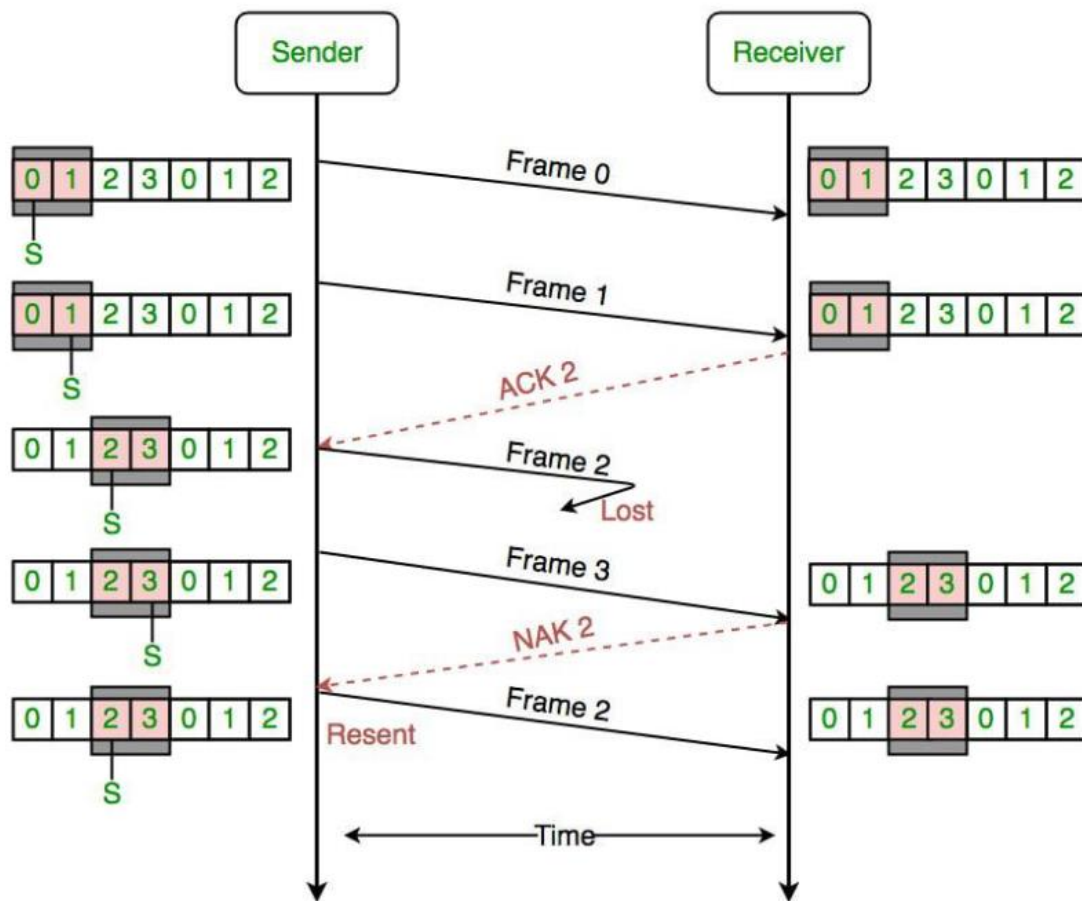


Рисунок 3. Диаграмма работы протокола Selective Repeat [2].

Отправитель в SR ведёт отдельный таймер для каждого отправленного, но ещё не подтверждённого кадра. Когда таймер какого-то кадра истекает, повторно передаётся только этот кадр. Окно сдвигается вправо, когда подтверждён кадр с номером `send_base`; после этого проверяется, не подтверждены ли уже следующие кадры по порядку, и окно сдвигается сразу через всю непрерывную подтверждённую последовательность. В результате, даже если в окне есть «дырка» (неподтверждённый кадр в середине), отправитель продолжает передавать новые кадры, попадающие в правую часть окна, а получатель продолжает буферизовать их. Как только дырка закрывается повторной передачей, вся накопленная последовательность доставляется выше по стеку одним махом.

Такая схема даёт очевидный выигрыш по объёму трафика: при потере кадра отправитель повторяет ровно один кадр, а не всю пачку. Однако платой за это становится более сложная реализация. Отправителю нужен набор таймеров – по одному на каждый неподтверждённый кадр. Получателю нужен буфер для хранения кадров, пришедших вне очереди. Кроме того, возникает проблема неоднозначности порядковых номеров, более серьёзная, чем в GBN: при циклическом переходе номеров получатель может спутать новый кадр с повторно переданным старым. Чтобы этого не происходило, размер окна SR ограничивается величиной $W \leq M/2$, где $M = 2^k$ – размер пространства номеров. Это ограничение вдвое строже, чем в GBN, потому что в SR одновременно «в полёте» могут находиться как новые кадры, так и повторно переданные, а также буферизованные у получателя.

4. Сравнение и метрики

Из выше сказанного следует основное практическое различие между двумя протоколами.

Selective Repeat эффективнее использует пропускную способность канала при наличии ошибок, поскольку не передаёт лишние кадры

повторно. Go-Back-N выигрывает простотой реализации и меньшим объёмом служебного состояния, что делает его предпочтительным в каналах с малой вероятностью ошибок или при малых размерах окна. При больших p и больших W разница в эффективности становится драматической – *GBN* может создавать в разы больше повторного трафика, чем *SR*, что и должно быть подтверждено в экспериментальной части работы.

Для количественного сравнения вводится метрика накладных расходов H , показывающая, какую долю от полезного трафика составляет повторная передача. Если обозначить T – общее число передач *DATA*-кадров (включая повторные), N – число полезно доставленных кадров, $R = T - N$ – число повторных передач, то:

$$H = \frac{R}{N} \cdot 100\% = \frac{T - N}{N} \cdot 100\%.$$

Связанный с ней коэффициент эффективности k показывает долю полезных передач от общего их числа:

$$k = \frac{N}{T} = \frac{1}{1 + \frac{H}{100}}.$$

Если потерь нет, $T = N$, $H = 0$, $k = 1$. Если каждый кадр передаётся в среднем дважды, $H = 100\%$, $k = 0.5$. В работе именно эти две метрики используются для сравнения протоколов при разных значениях p и W .

Наконец, приведём аналитические оценки H для обоих протоколов, с которыми будут сопоставляться экспериментальные данные. Обозначим $q = 1 - p$ – вероятность успешной передачи одного кадра. Для *Selective Repeat* каждая передача кадра независима и имеет геометрическое распределение числа попыток, поэтому среднее число передач на один успешно доставленный кадр равно $1/q$, а накладные расходы составляют:

$$H_{SR} = \frac{p}{1 - p} \cdot 100\%.$$

Важно, что эта формула не зависит от размера окна W : при потерях каждого кадра в отдельности окно влияет на параллелизм и задержку, но не

на объём повторного трафика.

Для *Go-Back-N* картина иная. За один «раунд» передаётся окно из W кадров. Число кадров X , на которое сдвинется левая граница окна до первой ошибки, имеет математическое ожидание:

$$EX = q + q^2 + \dots + q^W = \frac{q(1 - q^W)}{p},$$

и, следовательно, среднее число попыток на один доставленный кадр равно W/EX . Отсюда:

$$H_{GBN} \approx \left(\frac{W}{EX} - 1 \right) \cdot 100\%.$$

При $W = 1$ обе формулы дают одинаковый результат: и *GBN*, и *SR* вырождаются в протокол с остановкой и ожиданием (*Stop-and-Wait*), и накладные расходы у них совпадают. При $W > 1$ и $p > 0$ значение H_{GBN} растёт и с ростом p , и с ростом W .

Это качественное предсказание теории и будет сопоставлено с результатами имитационного моделирования.

ИМИТАЦИОННАЯ МОДЕЛЬ И РЕАЛИЗАЦИЯ

Имитационная модель системы передачи данных реализована на языке *Python 3* в среде разработки *PyCharm*. Программа построена по модульному принципу: каждый компонент системы – отправитель, получатель, канал связи и управляющая логика эксперимента – вынесен в отдельный файл и взаимодействует с остальными через стандартизованные интерфейсы. Такой подход упрощает отладку, позволяет независимо тестировать отдельные части модели и делает код легко расширяемым. Обмен данными между отправителем и получателем осуществляется через потокобезопасные очереди, играющие роль канала связи, а параллельная работа обеих сторон обеспечивается средствами стандартного модуля *threading*.

Канал связи моделируется классом *MsgQueue*, который инкапсулирует стандартную очередь *queue.Queue* и реализует имитацию потерь. При попытке поместить кадр в очередь с заданной вероятностью *loss_probability* кадр «теряется» и в очередь не попадает, что соответствует поведению реального канала с помехами. Модель использует два независимых канала: один для передачи кадров данных от отправителя к получателю и второй – для передачи подтверждений в обратном направлении. Такое разделение позволяет, во-первых, корректно учитывать потери в каждом направлении отдельно, а во-вторых, задавать их независимо: по умолчанию потери АСК отключены, что согласуется с классической аналитической моделью и позволяет изолировать влияние ошибок именно прямого канала. Кадр описывается классом *Packet*, который хранит порядковый номер, полезную нагрузку, признаки принадлежности к подтверждениям, номер подтверждаемого кадра и время отправки, используемое для контроля таймаутов.

Логика отправителя и получателя для обоих протоколов сосредоточена в классах *Sender* и *Receiver*, каждый из которых работает в отдельном потоке, что отражает асинхронный характер взаимодействия в реальной сети. Отправитель ведёт окно передачи, храня базовый номер *base*, номер

следующего кадра *next_seq_num* и словарь неподтверждённых кадров *unacked*, и циклически выполняет четыре действия: отправку новых кадров, если в окне есть место; обработку всех доступных подтверждений; проверку тайм-аутов; проверку условия завершения передачи. Счётчик *total_sent* фиксирует общее число попыток передачи, включая повторные, и используется для вычисления метрик эффективности.

Для протокола *Go-Back-N* подтверждения трактуются как кумулятивные: при получении *ACK* с номером *ack_num* отправитель удаляет из *unacked* все кадры с номерами до *ack_num* включительно и сдвигает *base*; таймер привязан к базовому кадру, и по его истечении повторно передаются все кадры от *base* до *next_seq_num* - 1, что и соответствует названию протокола.

Для протокола *Selective Repeat* подтверждения индивидуальны: при получении *ACK* удаляется только соответствующий кадр, а окно сдвигается вперёд через максимальный непрерывный подтверждённый префикс; таймер ведётся для каждого неподтверждённого кадра отдельно, и повторная передача затрагивает только тот кадр, чей таймер истёк.

Логика получателя также различается для двух дисциплин.

В режиме *GBN* получатель принимает только кадры, номер которых совпадает с ожидаемым, продвигает ожидаемый номер вперёд и отправляет кумулятивное подтверждение; кадры, пришедшие вне очереди, отбрасываются, а повторно отправляется *ACK* для последнего корректно принятого кадра.

В режиме *SR* получатель буферизует кадры, пришедшие вне очереди, отправляет на каждый из них индивидуальное подтверждение и при получении ожидаемого кадра доставляет сразу всю накопленную непрерывную последовательность. Завершение передачи фиксируется событием *threading.Event*, сигнализирующим о доставке всех кадров.

Управляющий модуль *experiment.py* запускает один прогон модели: создаёт каналы связи, конфигурирует отправителя и получателя с заданными

параметрами (*protocol*, *window_size*, *timeout*, *loss_probability*, *total_packets*), запускает оба потока, ожидает завершения и собирает статистику – общее число отправленных кадров, время передачи, накладные расходы N и коэффициент эффективности k . На верхнем уровне модуль *main.py* перебирает все комбинации протоколов, размеров окна и вероятностей ошибки, для каждой комбинации выполняет заданное число независимых прогонов, усредняет результаты и сохраняет их вместе со стандартными отклонениями в файл *results.csv*. Для воспроизводимости экспериментов генератор псевдослучайных чисел инициализируется фиксированным начальным значением. Построение графиков выполняет отдельный модуль *plot_results.py*, который читает сохранённый CSV и формирует семейства кривых $N(p)$, $k(p)$ и $t(p)$, а также график сопоставления экспериментальных данных с аналитическими оценками. Полный исходный код модели, инструкция по запуску и файлы с результатами экспериментов размещены в открытом репозитории [3].

Для проведения экспериментов использовались следующие значения параметров.

Количество передаваемых информационных кадров фиксировано и равно $N = 500$; число независимых прогонов для каждой комбинации параметров составляет 10, что позволяет получить выборочные средние и стандартные отклонения.

Размер скользящего окна принимает значения $W \in \{1, 2, 4, 8\}$, что соответствует степеням двойки и удобно для сопоставления с теорией; случай $W = 1$ соответствует вырождению протокола в схему с остановкой и ожиданием.

Вероятность потери кадра варьируется в диапазоне $p \in \{0.00, 0.05, 0.10, 0.15, 0.20, 0.25, 0.30\}$.

Время ожидания подтверждения установлено равным *timeout* = 0.05 секунды – значение, достаточное для оборота кадра через очередь при

данной конфигурации, но существенно меньшее, чем полное время передачи всего объёма данных, что даёт возможность срабатывания тайм-аутов при потерях.

Потери подтверждений отключены (*ack_loss_probability* = 0), что изолирует влияние ошибок прямого канала и согласуется с допущениями классической аналитической модели.

Аварийный тайм-аут одного прогона установлен равным 120 секундам, что заведомо больше времени завершения передачи при выбранных параметрах.

РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТА

1. Зависимость накладных расходов от вероятности ошибок

Основной метрикой сравнения протоколов служат накладные расходы H , показывающие, какую долю от полезного трафика составляет повторная передача кадров. На рисунке 4 представлены зависимости $H(p)$ для *Go-Back-N* и *Selective Repeat* при размерах окна $W \in \{1, 2, 4, 8\}$. Графики построены по усреднённым данным десяти независимых прогонов для каждой комбинации параметров при фиксированном числе передаваемых кадров $N = 500$.

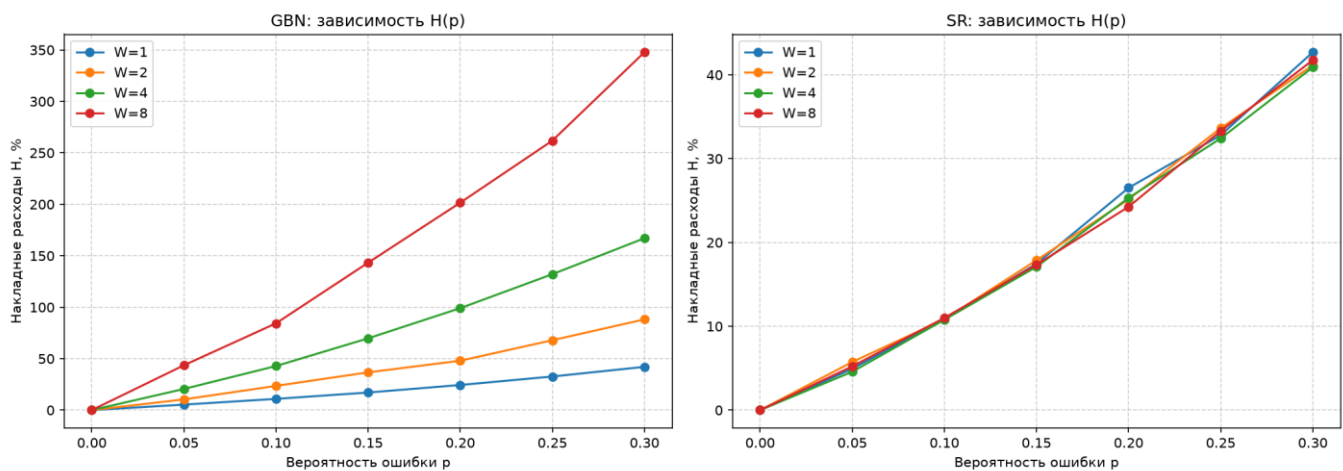


Рисунок 4. Накладные расходы *Go-Back-N* (слева) и *Selective Repeat* (справа) в зависимости от вероятности ошибки при различных размерах скользящего окна.

На левой части рисунка 4, соответствующей протоколу *Go-Back-N*, отчётливо видно, что накладные расходы быстро возрастают как с увеличением вероятности ошибки p , так и с ростом размера окна W . При фиксированном $p = 0.30$ значения H_{GBN} составляют приблизительно 42% при $W = 1$; 88% при $W = 2$; 167% при $W = 4$ и 348% при $W = 8$. Такое поведение объясняется природой протокола: потеря единственного кадра внутри большого окна приводит к повторной передаче всей неподтверждённой части окна, и чем больше W , тем больше «лишних» кадров передаётся заново. При малых p потери редки, и накладные расходы остаются умеренными даже для больших окон, однако с ростом вероятности

ошибки различие между кривыми становится драматическим.

Правая часть рисунка 4, соответствующая *Selective Repeat*, демонстрирует принципиально иную картину. Все четыре кривые для $W = 1, 2, 4, 8$ практически совпадают, а накладные расходы монотонно растут с вероятностью ошибки. При $p = 0.30$ значения H_{SR} равны 42.7%, 41.1%, 40.9% и 41.8% для $W = 1, 2, 4, 8$ соответственно – расхождение между кривыми лежит в пределах статистической погрешности. Это прямое экспериментальное подтверждение теоретического вывода о том, что для *Selective Repeat* накладные расходы не зависят от размера окна: протокол повторяет только действительно потерянные кадры, и окно влияет лишь на параллелизм и задержку, но не на объём повторного трафика. Именно это делает SR существенно более предпочтительным при работе в каналах с заметной вероятностью ошибок.

Оценим статистическую значимость наблюдаемого расхождения точнее. В *Selective Repeat* число попыток передачи одного кадра имеет геометрическое распределение с параметром $q = 1 - p$, поэтому стандартное отклонение накладных расходов в одном прогоне из $N = 500$ кадров составляет:

$$\sigma = \sqrt{\frac{p}{q^2 N}} \cdot 100\%.$$

Стандартная ошибка среднего по $m = 10$ независимым прогонам равна $\sigma/\sqrt{m}$, а соответствующий 95%-й доверительный интервал:

$$\pm 1.96 \frac{\sigma}{\sqrt{m}}.$$

При $p = 0.30$ это даёт $\sigma \approx 3.5$ процентных пункта и 95%-й интервал ± 2.2 процентных пункта. Наблюдаемый разброс между кривыми SR при $p = 0.30$ (от 40.9 % до 42.7 %, то есть менее 2 процентных пунктов) целиком укладывается в этот интервал. Следовательно, расхождение между кривыми

SR для разных W не является систематическим и полностью объясняется статистическим разбросом, что согласуется с теоретическим выводом о независимости H_{SR} от размера окна. Для *Go-Back-N* аналогичная аналитическая оценка дисперсии затруднена из-за коррелированных групповых повторных передач внутри окна; консервативная верхняя оценка 95%-го интервала при $p = 0.30$ составляет $\pm 3-4$ процентных пункта, что на порядок меньше наблюдаемого различия между *GBN* и *SR* (например, 41.8 % против 347.8 % при $W = 8$). Поэтому все дальнейшие выводы о различии протоколов опираются на эффекты, многократно превышающие ширину доверительных интервалов.

2. Зависимость коэффициента эффективности от вероятности ошибок

Коэффициент эффективности $k = N/T$ связан с накладными расходами соотношением $k = \frac{1}{1 + \frac{H}{100}}$, поэтому его поведение качественно повторяет поведение $H(p)$, но в более удобной для интерпретации форме: k показывает долю полезных передач от общего их числа, а $1 - k$ – долю повторных.

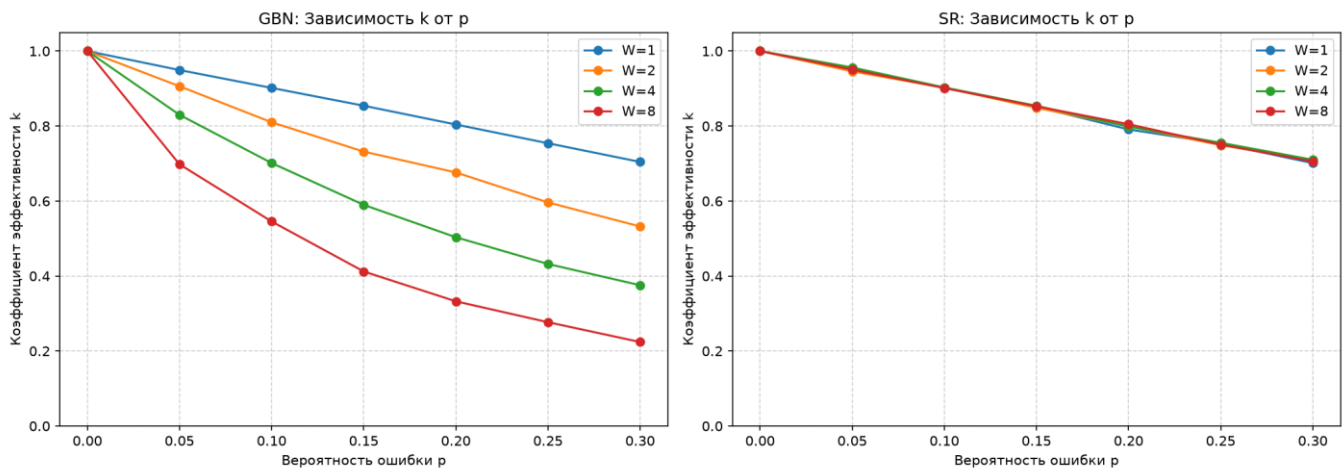


Рисунок 5. Коэффициент эффективности *Go-Back-N* (слева) и *Selective Repeat* (справа) в зависимости от вероятности ошибки при различных размерах окна.

На рисунке 5 видно, что для *GBN* при $W = 8$ и $p = 0.30$ коэффициент эффективности падает до $k \approx 0.22$, то есть лишь 22% всех переданных

кадров являются полезными, а остальные 78% – повторные. Для того же значения p в SR при любом размере окна k составляет около 0.70–0.71: полезными оказываются примерно 70% передач. При $p = 0$ оба протокола дают $k = 1.0$, что соответствует отсутствию потерь и полному отсутствию повторных передач.

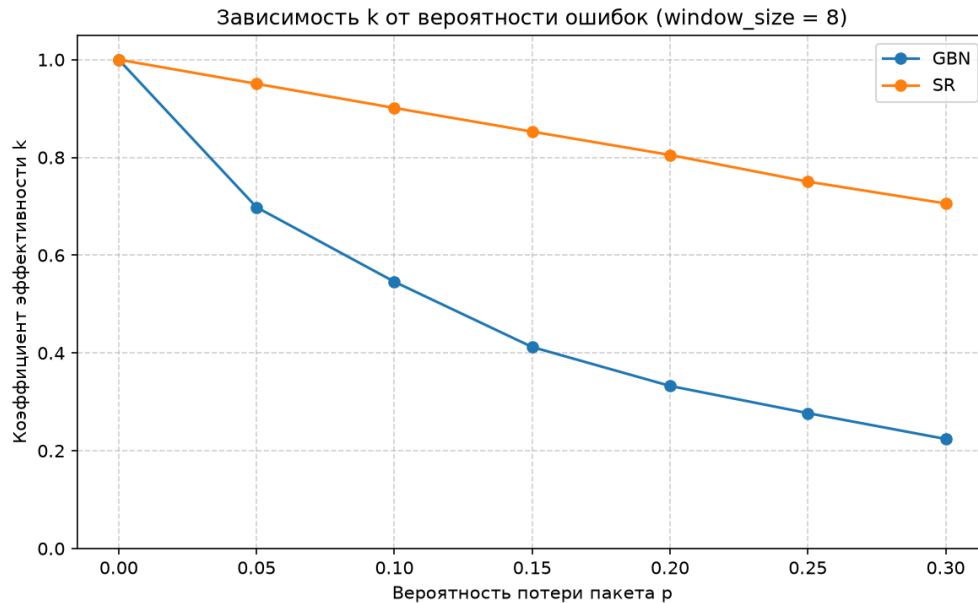


Рисунок 6. Сравнение коэффициента эффективности GBN и SR при размере окна $W = 8$.

Прямое сравнение двух протоколов при $W = 8$ (рисунок 6) показывает, что при $p = 0.30$ выигрыш SR в коэффициенте эффективности превышает трёхкратный: $k_{SR} \approx 0.71$ против $k_{GBN} \approx 0.22$. Особенно важно, что кривые начинают расходиться уже при $p = 0.05$: при этой вероятности ошибки $k_{GBN} \approx 0.70$, тогда как $k_{SR} \approx 0.95$. Это означает, что даже при сравнительно «чистом» канале $Go-Back-N$ при большом окне начинает тратить значительную долю пропускной способности впустую.

3. Зависимость времени передачи от вероятности ошибок

Время передачи t – вспомогательная метрика, отражающая полное время от начала работы отправителя до получения подтверждения на последний кадр. Абсолютные значения t зависят от производительности машины и от выбранного значения тайм-аута, поэтому эта метрика

используется не для сопоставления с теорией, а для качественной оценки задержки.

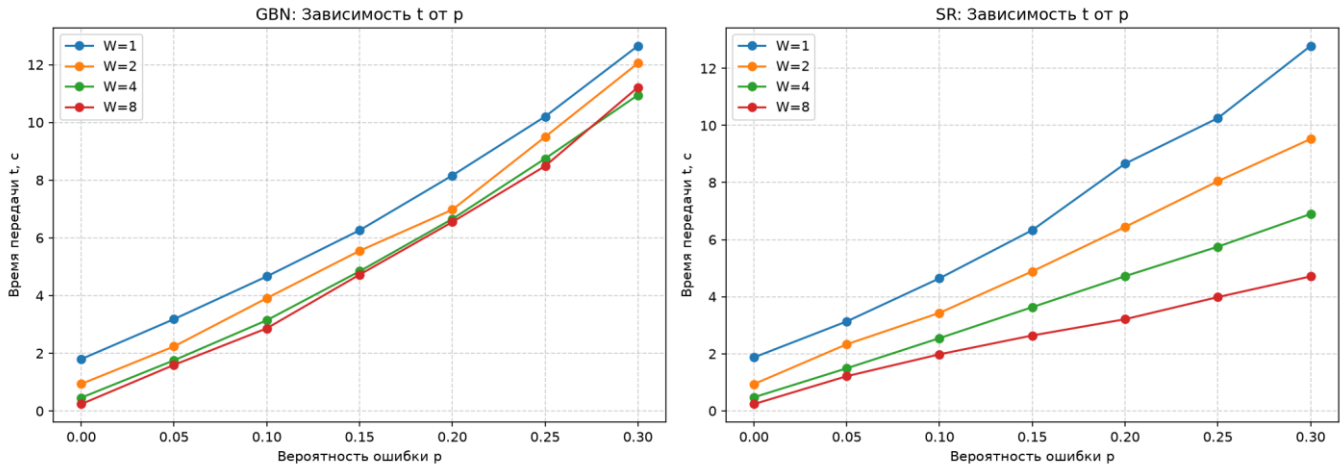


Рисунок 7. Время передачи *Go-Back-N* (слева) и *Selective Repeat* (справа) в зависимости от вероятности ошибки при различных размерах окна.

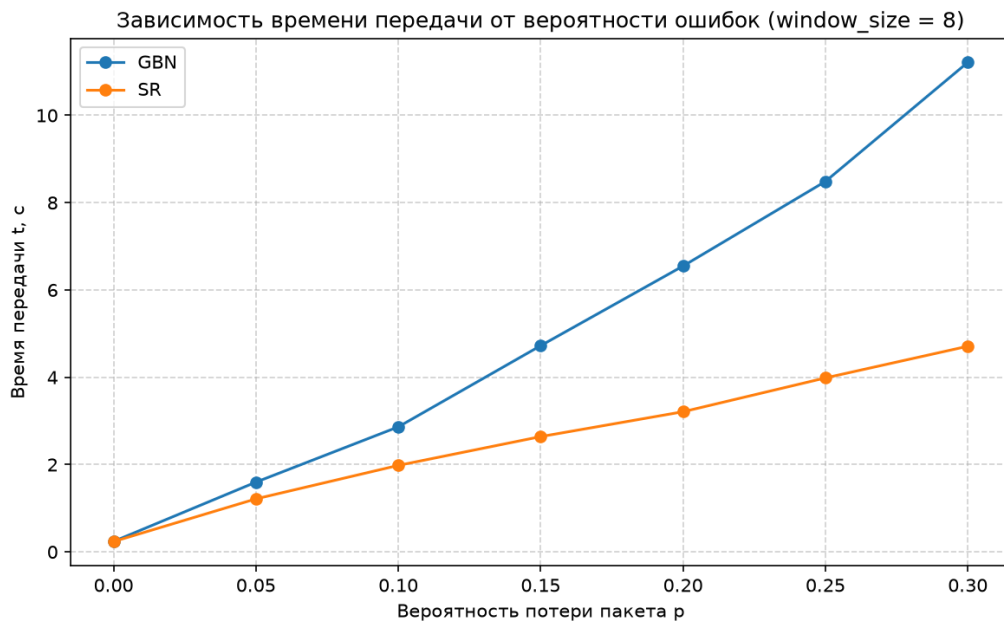


Рисунок 8. Сравнение времени передачи *GBN* и *SR* при $W = 8$.

Из рисунков 7 и 8 видно, что при $p = 0$ оба протокола завершают передачу 500 кадров примерно за одинаковое время (для $W = 8$ это около 0.24 секунды). С ростом вероятности ошибки время передачи увеличивается у обоих протоколов, но у *GBN* – значительно быстрее. При $p = 0.30$ и $W = 8$ передача занимает около 11.2 секунды при *GBN* и 4.7 секунды при *SR* – разница более чем двукратная. Для *GBN* рост времени связан с двумя

факторами: во-первых, на каждую повторную передачу пачки кадров уходит дополнительное время; во-вторых, таймеры срабатывают чаще, и отправитель дольше остаётся в ожидании. В *SR* повторяется только один кадр, поэтому время растёт медленнее.

Также важно отметить, что при фиксированном p увеличение размера окна сокращает время передачи. Это хорошо видно на рисунке 7: при $p = 0.30$ *GBN* при $W = 1$ передаёт данные за 12.6 секунды, а при $W = 8$ – за 11.2 секунды; для *SR* различие ещё заметнее – 12.8 и 4.7 секунды соответственно. Увеличение окна позволяет держать больше кадров «в полёте» и эффективнее использовать доступную пропускную способность, что согласуется с интуитивными соображениями и с принципом конвейеризации.

4. Сравнение с аналитическими оценками

Для верификации корректности реализации экспериментальные данные были сопоставлены с классическими аналитическими формулами $H_{SR} = \frac{p}{1-p} \cdot 100\%$ и $H_{GBN} \approx \left(\frac{W}{\mathbb{E}X} - 1\right) \cdot 100\%$, выведенными в предположении раундовой модели передачи.

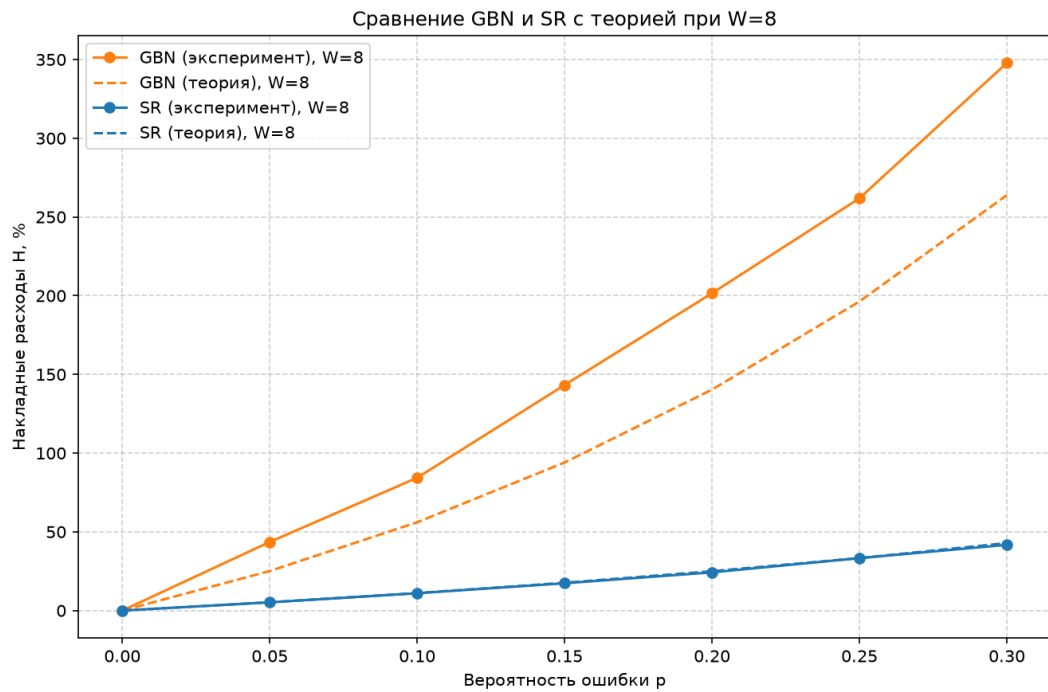


Рисунок 9. Сравнение экспериментальных и аналитических оценок накладных расходов для *GBN* и *SR* при $W = 8$.

Из рисунка 9 видно, что экспериментальные точки *Selective Repeat* практически идеально ложатся на теоретическую кривую. Например, при $p = 0.10$ эксперимент даёт $H_{SR} = 11.0\%$, при теоретических 11.1% ; при $p = 0.20$ – 24.3% против 25.0% ; а при $p = 0.30$ – 41.8% против 42.9% . Такое совпадение в пределах доверительных интервалов подтверждает корректность реализации *SR*: поскольку в этом протоколе каждая повторная передача вызвана только конкретной потерей, а не групповым сбросом, раундовая и непрерывная модели дают одинаковый результат.

Для *Go-Back-N* наблюдается систематическое расхождение. Экспериментальные значения H_{GBN} оказываются выше аналитических оценок, причём отношение $H_{\text{эксперимент}}/H_{\text{теория}}$ уменьшается с ростом p : примерно 1.74 при $p = 0.05$; 1.51 при $p = 0.10$; 1.33 при $p = 0.25$ и 1.32 при $p = 0.30$. Это расхождение не является следствием ошибки в коде – оно объясняется различием между раундовой идеализацией, использованной при выводе формул, и непрерывной моделью передачи, реализованной в имитации.

Причиной этому является то, что аналитическая формула предполагает: отправитель передаёт окно целиком, дожидается результата всех кадров этого раунда, затем сдвигает окно и только после этого начинает новый раунд. В реализованной имитационной модели передача идёт непрерывно: как только окно сдвигается, отправитель немедленно передаёт новые кадры, попадающие в освободившиеся позиции окна. Если внутри окна теряется кадр, отправитель по истечении тайм-аута повторяет не только потерянный кадр, но и все новые кадры, попавшие в окно за это время. В раундовой модели таких новых кадров не было бы, поэтому формула даёт заниженную оценку. Именно этим объясняется и систематический характер расхождения, и его уменьшение с ростом p : при больших p потери происходят так часто, что окно редко успевает «заполниться» новыми кадрами, и разница между моделями сглаживается.

Для *Selective Repeat* подобного расхождения не возникает, поскольку каждая повторная передача затрагивает только один конкретный кадр, независимо от того, сколько новых кадров уже отправлено. Именно поэтому *SR* совпадает с теорией, а *GBN* – нет. Это различие, в свою очередь, служит косвенным подтверждением корректности реализации: код воспроизводит ожидаемое поведение каждой дисциплины, а наблюдаемое отклонение *GBN* носит системный, объяснимый характер.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были реализованы имитационные модели двух протоколов автоматического запроса повторной передачи – *Go-Back-N* и *Selective Repeat* – и проведено экспериментальное исследование их эффективности в канале с потерями кадров при различных размерах скользящего окна. Всего было выполнено 560 независимых прогонов: 2 протокола $\times$ 4 размера окна $\times$ 7 значений вероятности ошибки $\times$ 10 повторов. По результатам построены семейства графиков накладных расходов, коэффициента эффективности и времени передачи, а также выполнено сопоставление с аналитическими оценками.

Проведённые эксперименты подтвердили основные теоретические предсказания. При отсутствии потерь ($p = 0$) оба протокола обеспечивают нулевые накладные расходы и единичный коэффициент эффективности, поскольку каждая передача оказывается успешной с первой попытки. При размере окна $W = 1$ обе дисциплины вырождаются в протокол с остановкой и ожиданием и дают практически совпадающие результаты: например, при $p = 0.30$ накладные расходы составляют 42.1% для *GBN* и 42.7% для *SR*, что лежит в пределах статистической погрешности. С ростом вероятности ошибки накладные расходы увеличиваются у обоих протоколов, однако характер этого роста существенно различается.

Главный вывод работы состоит в том, что эффективность *Selective Repeat* практически не зависит от размера окна, тогда как для *Go-Back-N* наблюдается резкий рост накладных расходов с увеличением W . При $p = 0.30$ значения H_{SR} для $W = 1, 2, 4, 8$ лежат в диапазоне 40.9–42.7%, тогда как H_{GBN} увеличивается с 42.1% до 347.8% – почти восьмикратный рост. Это означает, что при наличии ошибок в канале увеличение окна для *GBN* не только не помогает, но и приводит к стремительному росту повторного трафика. Причина в том, что потеря одного кадра внутри большого окна заставляет отправителя повторно передавать всю неподтверждённую часть окна, тогда как *SR* повторяет только конкретный потерянный кадр.

Сравнение с аналитическими формулами показало, что экспериментальные значения H_{SR} совпадают с теоретическими в пределах доверительных интервалов, что подтверждает корректность реализации. Для *GBN* наблюдается систематическое расхождение: экспериментальные значения на 30–75% выше теоретических в зависимости от p . Это расхождение объясняется различием между раундовой моделью, использованной при выводе формул, и непрерывной моделью передачи, реализованной в имитации. В непрерывной модели отправитель успевает передать новые кадры, пока ожидает тайм-аут для потерянного кадра, и все эти новые кадры впоследствии попадают в повторную передачу. Именно поэтому реализованная модель даёт более консервативные оценки накладных расходов, чем классическая аналитическая формула, и это делает её ближе к поведению реальных *ARQ*-протоколов, в частности *TCP*.

С точки зрения практических рекомендаций проведённое исследование позволяет сформулировать следующие выводы. Протокол *Selective Repeat* существенно предпочтительнее при работе в каналах с заметной вероятностью ошибок и при использовании больших размеров окна, поскольку он обеспечивает минимальный объём повторного трафика и слабо зависит от W . Платой за эту эффективность является более сложная реализация: необходимость индивидуальных таймеров для каждого кадра, буферизации внеочередных кадров у получателя и более строгого ограничения на размер окна ($W \leq M/2$ против $W \leq M - 1$ у *GBN*). Протокол *Go-Back-N* выгоден в каналах с малой вероятностью ошибок или при малых размерах окна: его реализация существенно проще, а избыточность передачи остаётся приемлемой. При проектировании реального протокола выбор между двумя дисциплинами должен учитывать не только накладные расходы, но и задержку, требуемую пропускную способность, объём доступной памяти и допустимую сложность реализации.

Ограничения проведённого исследования связаны с принятой моделью канала: потери считаются независимыми (модель Бернулли), потери

подтверждений отключены для изоляции влияния прямого канала, время моделируется непрерывно с реальными таймерами, а все кадры имеют одинаковую длину. Эти упрощения не влияют на главные качественные выводы, но должны учитываться при переносе результатов на реальные сети, где возможны пакетные потери, задержки, переупорядочивание и асимметрия прямого и обратного каналов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Go-Back-N Protocol ARQ: [сайт]. – URL:

<https://binaryterms.com/go-back-n-protocol.html>

(дата обращения: 22.09.2026).

2. Difference Between Stop and Wait, GoBackN and Selective Repeat: [сайт]. –

URL:

<https://www.geeksforgeeks.org/computer-networks/difference-between-stop-and-wait-gobackn-and-selective-repeat/>

(дата обращения: 22.09.2026).

3. Репозиторий с кодом реализации лабораторной работы: [сайт]. – URL:

<https://github.com/AnastasiaHudina/Computer-Networks/tree/main/lab1>

(дата обращения: 22.09.2026).